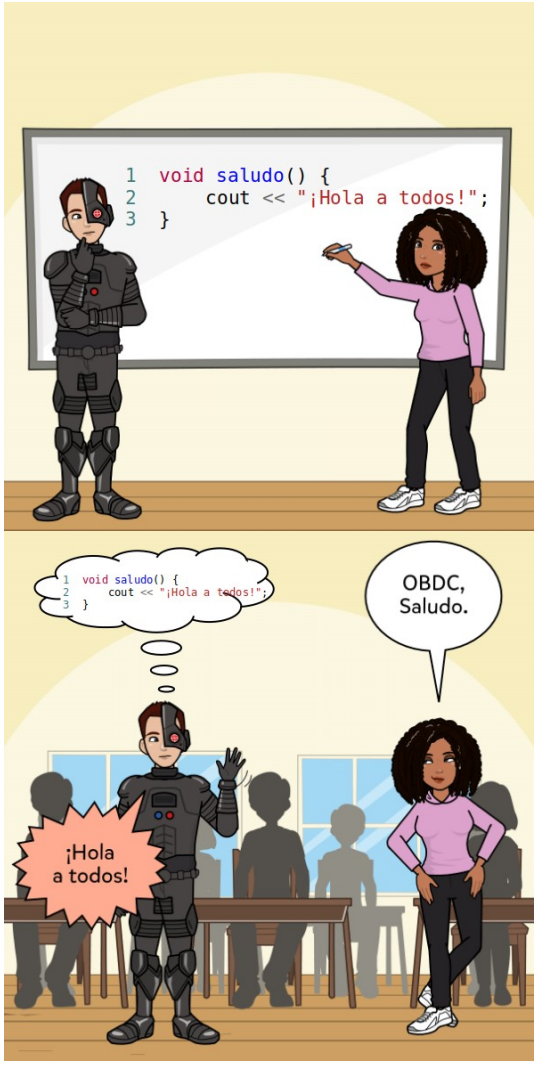
Lectura 6: Funciones

Introducción.

En el mundo de la programación, las funciones desempeñan un papel fundamental, ya que son *bloques de código* reciclables que permiten a los desarrolladores encapsular y ejecutar tareas específicas de manera eficiente. Dentro de este contexto, es crucial comprender también el concepto de variables locales y globales, ya que estas determinan el alcance y el comportamiento de los datos utilizados en las funciones. Exploraremos en profundidad estos elementos, proporcionando ejemplos prácticos y ejercicios que ayudarán a los lectores a afianzar la comprensión de ellos en el mundo de la programación.

Funciones en Programación

Las funciones son bloques de *código reutilizables* que realizan una tarea específica y bien definida. Estas permiten a los desarrolladores organizar y modular el código, lo que facilita su posterior lectura, mantenimiento y depuración. Una función típica consta de un nombre, parámetros de entrada (si son necesarios) y un conjunto de instrucciones que se ejecutan cuando se hace el llamado a la función. Las funciones pueden también devolver valores si es necesario.



Exploraremos también dos tipos de variables que se utilizan en el contexto de las funciones: las variables locales y las variables globales. Entender la diferencia entre ellas es fundamental para comprender cómo se gestionan los datos en un programa.

Además, aprenderemos que existen dos tipos de funciones: las predefinidas y las definidas por el usuario. En el primer caso, es necesario el empleo de *librerías* que

permite el acceso a las funciones específicas. En el segundo caso, de las funciones definidas por el usuario, se tiene la siguiente estructura:

```
TipoRetorno NombreFuncion (parametro1, parametro2,...)
// Bloque de instrucciones

```

Vamos a practicar escribiendo una función que presenta un saludo en el *dispositivo de salida* estándar empleando el lenguaje C++:

```
// Declaración de una función
void saludo() {
    cout << "¡Hola a todos!";
}
```

En el código anterior podemos ver lo siguiente:

- `void` indica al intérprete que la función no retorna ningún valor.
- `saludo` es el nombre que le da el usuario a la función. Cuando se requiera el uso de la función, se llama usando este nombre.
- `()` los paréntesis vacíos indican al intérprete que no hay parámetros o argumentos para la función.
- `{ }` las llaves delimitan el bloque de instrucciones que forman el cuerpo de la función.

Para llamar una función se emplea su nombre como se puede ver en el código siguiente:

```
#include <iostream>       //Librería con funciones de impresión en pantlla
using namespace std;      //Selecciona una sección particular de la librería.

// Declaración de una función
void saludo() {
      cout << "¡Hola a todos!";
}

int main() {
    saludo();             //Lamado a la función
    return 0;
}
```



Emplea este enlace programiz para copiar el programa y ejecutarlo.

Parámetros de una función

Cuando se define una función, se pueden incluir parámetros de entrada que actúan como variables dentro del cuerpo de la misma. Estos parámetros permiten a la función recibir datos externos y utilizarlos en su secuencia interna de instrucciones. En C++, se declaran entre los paréntesis que siguen al nombre de la función. Cuando se llama a la función, se deben proporcionar los valores correspondientes a cada uno de los parámetros definidos.

Por ejemplo, considera la siguiente función que calcula el área de un círculo:

```cpp
#include <math.h>                //Librería matemática con las funciones predefinidas atan2 y pow
#include <iostream>              //Librería con funciones predeterminadas de lectura y escritura

using namespace std;

double Circulo_Area(double r) {  //Declaración de la función con argumento de entrada
  double PI = 2.*atan2(1,0);     //Declaración de una variable local PI
  return PI * pow(r, 2);         //Valor que retorna la función
}

int main() {
    double radio;
    cout << "Digite el radio del círculo (unidades arbitrarias, u): ";
    cin >> radio;
    double area = Circulo_Area(radio);
    cout << "El área del círculo es: " << area << "u²." << endl;
    return 0;
}
```

Emplea este enlace programiz para copiar el programa y ejecutarlo.

En el ejemplo podemos notar:

- La inclusión de librerías en C++ se hace mediante la instrucción `#include`, con el nombre de la librería entre `<·>` . Una librería es un conjunto de funciones previamente definidas.
- La librería `math.h` incluye funciones matemáticas como la potenciación `pow` y la arcotangente `atan2`.
- El parámetro de la función se define entre los paréntesis como una variable que sólo existe en el bloque de la función, `(double r)`, es decir, fuera de ella es una variable desconocida. Este tipo de variables se denomina local.
- Cuando se llama la función, se debe agregar entre paréntesis los valores para los parámetros que se requieren para la correcta ejecución del código. Se puede relacionar de forma numérica ó mediante el valor de otra variable. En este caso `(radio)`.
- El valor que entrega la función, si lo hay, se establece mediante la sentencia `return`.

Variables Locales

Las variables locales son aquellas que se definen dentro de una función o bloque de código y sólo son accesibles dentro del ámbito de esa función o bloque. En otras palabras, éstas variables tienen sentido únicamente dentro de la función en la que se definieron y, por tanto, no pueden ser accedidas o modificadas desde fuera de esa función. Las variables locales se utilizan para almacenar *datos temporales* que son necesarios durante la ejecución de la función, y su alcance se limita al bloque de código en el que se definen. Es importante tener en cuenta que las variables locales se crean y se destruyen cada vez que se llama a la función, lo que significa que su valor se pierde una vez que la función ha terminado de ejecutarse.

En el ejemplo anterior, encontramos la variable `double PI` que se define dentro de un conjunto de instrucciones que son el cuerpo de la función y su existencia termina con la función, al igual que el parámetro `double r`.

Variables Globales



A diferencia de las variables locales, las variables globales son accesibles desde cualquier parte del programa. Estas variables se definen fuera de las funciones y tienen un ámbito general, lo que significa que pueden ser accedidas y modificadas por cualquier función del programa. Las variables globales se utilizan para almacenar datos que deben ser compartidos entre múltiples funciones o que deben mantenerse disponibles durante toda la *ejecución del programa*. Es importante tener en cuenta que el uso excesivo de variables globales puede llevar a problemas de *mantenimiento y escalabilidad del código*, ya que puede ser difícil seguir su valor en el flujo de datos y rastrear cómo se modifican las variables a lo largo del programa. Si quisiéramos calcular no sólo el área sino también el perímetro de un círculo, necesitamos usar el valor π en ambas funciones. Por tanto, que es más organizado tener acceso a este valor desde cualquier lugar del programa y no usarlo sólo de forma local. Este caso se puede analizar con el siguiente código:

```cpp
#include <math.h>          //atan2, pow
#include <iostream>        //cin, cout

using namespace std;

double PI;                 //Definición de una variable global PI

double Circulo_Area(double r) {
  return PI * pow(r, 2);
}

double Circulo_Perimetro(double r) {
  return 2. * PI * r;
}

int main() {
  PI = 2.*atan2(1,0);      //Asignación del valor a la variable PI
  double radio;
  cout << "Digite el radio del círculo (unidades arbitrarias, u): ";
  cin >> radio;
  double area = Circulo_Area(radio);
  double perimetro = Circulo_Perimetro(radio);
  cout << "El área del círculo es " << area
       << "u² y su perímetro es "<<perimetro<<"u."<<endl;
  return 0;
}

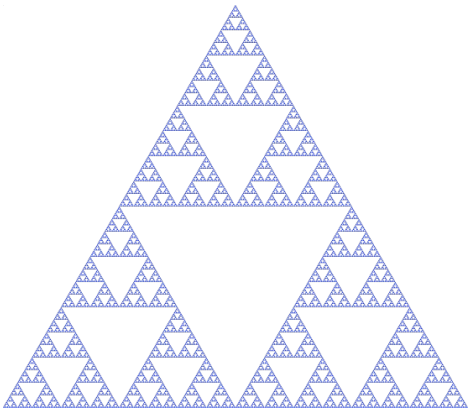
```

Emplea este enlace programiz para copiar el programa y ejecutarlo.

El triángulo de Sierpiński.

El triángulo de Siperpiński es un *fractal* que se puede dibujar con el siguiente algoritmo:

1. En un plano cartesiano grafique los vértices de un triángulo equilátero cuya base está sobre el eje horizontal y el vértice izquierdo está en el origen.
2. Seleccione al azar un punto dentro del triángulo como punto de partida.
3. Imprima las coordenadas x y y del punto.
4. Seleccione al azar uno de los vértices.
5. Tome el punto medio entre el punto y el vértice escogido.
6. Repita desde el punto 3.



Al graficar el conjunto de puntos se puede ver el fractal.

Tarea

1. Desarrolle el diagrama de flujo para dibujar el triángulo de Sierpiński.
2. Escriba un código en PYTHON que genere los puntos que pertenecen al triángulo de Sierpiński usando una función que retorne un número entero aleatorio entre uno y tres y otras funciones que dados dos puntos en un plano cartesiano retorne la coordenada x y y del punto medio respectivamente.
3. Elabore un glosario con las palabras en itálicas.